STM32CubeIDE and STM32CubeMX

Before continuing with any of the toolchain development below, please realize that the GUI version of STM32 development tools cover everything you need. As discussed in class, the GUI is a very productive mode of development, and you do not need the CLI based toolchain installed. But the GUI does obscure much of what is happening under the hood, and as such it can be difficult to debug problems if you become dependent on the GUI and don't understand what it is trying to do for you. Further, if you ever need to develop on another embedded platform outside the STMicroelectronics product portfolio you will find yourself learning everything from the beginning again. These are a couple advantages the GNU toolchain offers to you.

So that said, if you are inclined, and I would *strongly recommend you try to pursue the CLI based toolchain* (GNU toolchain in particular) in addition to the GUI, then the tips below will help you do so.

In the senior capstone lab, there will *not* be access to the GNU toolchain without substantial work from the administrators, so you will likely be limited to the STM32CubeIDE and STM32CubeMX GUI tools while working there. You should arrange to have access to your codebase via GitHub so that you can develop at home using either the CLI or the GUI, as well as during the times we meet in the capstone lab.

Bash or zsh Shell

If you are using Linux or MacOS then you already have access to a shell.

If you are using Windows you have a few options:

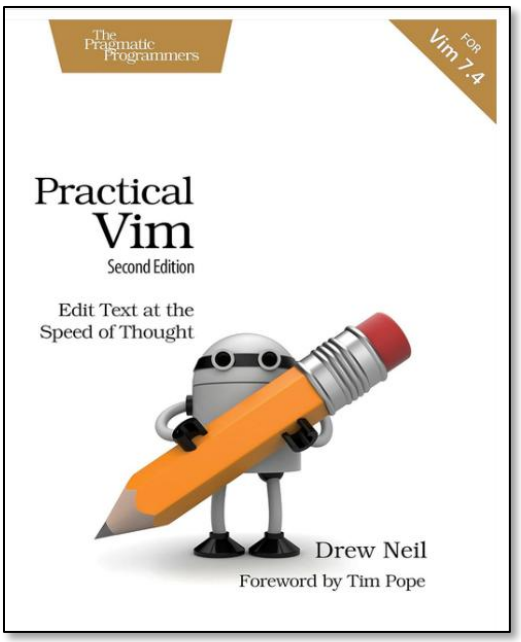
1. Install Windows Subsystem for Linux (WSL2)
2. Run a Linux virtual machine in VirtualBox
3. Install MSYS2
4. Grab yourself a Raspberry Pi and connect via SSH. You do NOT want the heavy GUI desktop.

These are unfortunately a bit out of scope for our course and there is insufficient time to cover tutorials in the lectures. Please use AI to guide you through installation of any one or more of the above options.

Editor

I use vim, but you can use emacs, neovim, Sublime, or VS Code. For most students I would recommend VS Code. It fully supports the LSP interface that lets you see definitions and navigate efficiently. You can use nano, but it is probably too lightweight to support the Language Server Protocol (LSP).

Several intrepid students expressed interest in my setup with vim and LSP. If you have not used vim before, you might find the following reference valuable. Warning, it is a very powerful editor but not for the faint of heart. Same with emacs. Both require a steep learning curve, but that learning will pay off tremendously if you plan to do any related work in the future.



The following book is what I consider a good introduction to vim and to some extent neovim. I recommend it highly.

LSP is "Language Server Protocol" and it enables any LSP compatible editor (like vim, emacs, etc.) to do things like autocomplete, go-to-definition, show code diagnostics and errors, and hover info — and *clangd* is the particular language server implementation for C/C++ that you'd be using with your editor in EE426/626.

For information about using LSP with your own editor:

microsoft.github.io/language-server-protocol
clangd.llvm.org (C embedded /ARM)
Neovim's `:help lsp` (also readable at neovim.io/doc)
VS Code's "Language Servers" extension-authoring docs

GNU Toolchain

Once you have a Linux / macOS shell, you can install the GNU toolchain. The tools are gcc, objdump, objcopy, openocd, etc. But in the case of ARM (as is the STM32) you will use a branch of the tool chain tailored specifically for the ARM processors. The tools are prefixed with 'arm-none-eabi-' meaning ARM is the target platform, 'none' is the vendor specific flavor of ARM, 'eabi' means Embedded Application Binary Interface which is how the compiler knows how to call functions and pass parameters on the stack optimized for bare metal embedded applications without a host operating system.

So, for example:

`arm-none-eabi-`**`gcc`**	C compiler / driver front-end
`arm-none-eabi-`**`g++`**	C++ compiler
`arm-none-eabi-`**`as`**	assembler
`arm-none-eabi-`**`ld`**	linker
`arm-none-eabi-`**`objcopy`**	convert object files (e.g. ELF -> .bin / .hex)
`arm-none-eabi-`**`objdump`**	disassemble / inspect object files
`arm-none-eabi-`**`gdb`**	debugger (may need gdb-multiarch on Debian / Ubuntu)
`arm-none-eabi-`**`size`**	report section sizes (.text / .data / .bss)
`arm-none-eabi-`**`nm`**	list symbols in an object file
`arm-none-eabi-`**`readelf`**	display ELF file structure / headers
`arm-none-eabi-`**`ar`**	create / manage static library archives
`arm-none-eabi-`**`strip`**	strip symbols / debug info from a binary

Remember: AI is your friend here; let it help you to understand LSP and also to install the toolchain !!

At the Ubuntu / Debian Command Line

```
sudo apt update
sudo apt install -y gcc-arm-none-eabi
sudo apt install -y binutils-arm-none-eabi
sudo apt install -y libnewlib-arm-none-eabi
sudo apt install -y gdb-multiarch
sudo apt install -y make
sudo apt install -y openocd
sudo apt install -y stlink-tools

wget https://www.segger.com/downloads/jlink/JLink_Linux_x86_64.deb
sudo apt install -y ./JLink_Linux_x86_64.deb
```

Notes:

JLink download URL needs to be checked and updated against segger.com/downloads/jlink before publishing. The SEGGER version numbers appear in that path, and it changes over time.

gdb-multiarch stands in for arm-none-eabi-gdb on Debian/Ubuntu.

openocd + stlink-tools are for flashing/debugging via ST-LINK, separate from the compiler toolchain itself.

At the macOS Command Line

On macOS we use a package manager called *Homebrew*. If you do not have it:

```
/bin/bash -c "$(curl -fsSL \
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then,

```
brew install --cask gcc-arm-embedded
brew install open-ocd
brew install stlink
```

Notes:

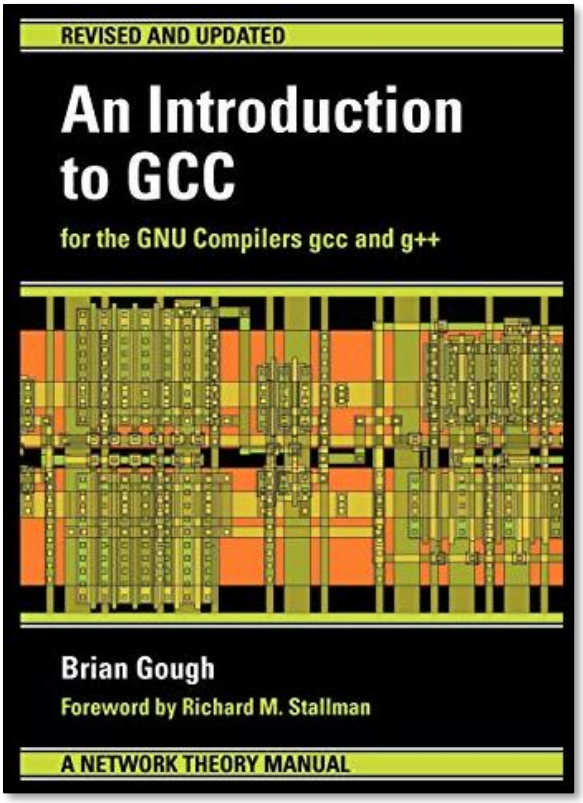
Get the Homebrew install command fresh from brew.sh rather than hardcoding it — it can change.

gcc-arm-embedded is a cask (prebuilt binary), not a formula; verify this is still the current Homebrew name/method shortly before publishing, since it's shifted before. Arm's own installer at developer.arm.com is the fallback if the cask lags.

SEGGER J-Link for Mac: no Homebrew formula — download the .pkg directly from segger.com/downloads/jlink and install manually, same caveat as the Linux .deb about checking the current version link.

GDB on macOS for ARM: the cask above should include arm-none-eabi-gdb; if not, it may need separate handling since GDB has extra macOS code-signing requirements that are not present on Linux.

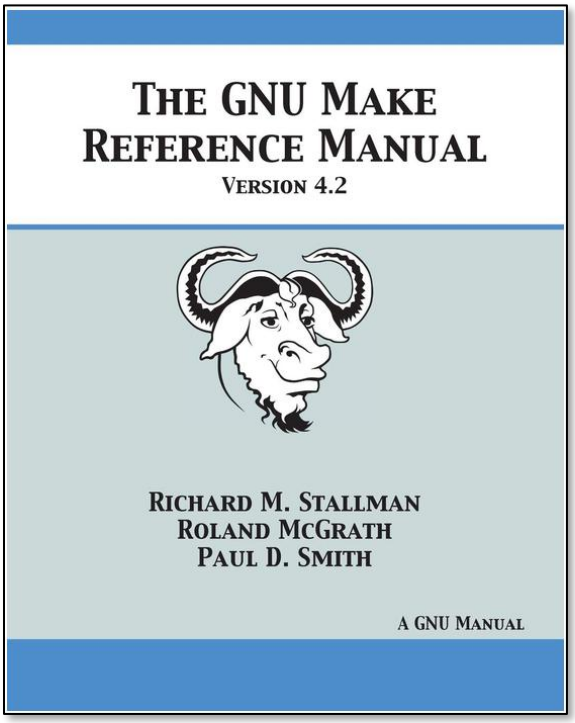
An Introduction to GCC



If you can find a copy of this text used, it is very helpful if you prefer written books over online documentation. It introduces you to many of the command options that we included with the gcc command when we ran it manually in class lecture.

It seems to have become a bit of a classic and as such the price is unreasonable and inexplicably high. I'm not sure why. But perhaps you can find a used copy for a reasonable price.

The GNU Make Reference Manual



This is a newer version of the book I showed in class. It can be had for a very reasonable price and should be free as a PDF download based on its source. This will help you understand how to create and maintain the Makefile in a CLI based tool chain.

How to Blink the LED on STM32 Development Board

The required textbook for the course (by Joseph Yiu) is a rather heavy hitting and dense coverage of the Cortex-M3/4 processors, not the STM32F446 chip itself. For that chip-specific information, you'll want to refer to the Reference Manual RM3090 that you can find at: https://www.st.com.

RM3090 Reference Manual – STM32F446xx advanced Arm-based 32-bit MCUs

If you'd like a gentler introduction that walks you through step-by-step in plain English how to set up the GNU tool chain and get the STM32 chip working for very simple projects, I'd highly recommend the following book by Warren Gay:



While it's a relatively simple text, written around the freshman level, it does get you up to speed quickly. And it does so without the crutch of a GUI-based IDE.

An excerpt from the text, on page 6:

No IDE

There was a conscious decision to choose for this book a development environment that was neutral to your desktop development platform of choice. There are a number of Windows-based IDE environments available, with varying licenses. But the IDEs change, licenses change, and their associated libraries change with time. The advantage of the given IDE is often discarded when the IDE and the operating system it runs upon change.

Using a purely open sourced approach has the advantage that you are shielded from all this version churn and burn. You can mothball all of your code and your support tools, knowing that they can all be restored to operation ten years from now, if required. Restoring licensed software, on the other hand, leaves you vulnerable to expired licenses or online sites that have gone dark.

This book develops projects based upon the following open sourced tools and libraries:

- gcc/g++ (GNU compiler collection: open sourced)
- make (GNU binutils: open sourced)
- libopencm3 (library: open sourced) – Not used in EE426/626
- FreeRTOS (library: open source and free for commercial use) – Not used in EE426/626

With this foundation, the projects in this book should remain usable long after you purchase the book. Further, it permits Linux, FreeBSD, and MacOS users – in addition to those using the Windows platform – to use this book ...

I could not have said the above any better than this author! It is another highly recommended book if you need to get up and off the ground quickly just to get your chip working with the CLI toolchain.